Relatório de Auditoria de Segurança — Gabarita (Corretor de Prova IA)

Data da auditoria: 31/08/2026

Escopo: monorepo pnpm — **apps/api** (backend Fastify), **apps/web** (frontend Next.js), **packages/db** (schema Drizzle ORM), **packages/env**, arquivos de configuração e deploy (.env, .env.example, docker-compose.dev.yml) e histórico Git.

Camada	Tecnologia detectada
Backend	Fastify + TypeScript (Vertical Slice Architecture, 1 arquivo = 1 caso de uso)
ORM / Banco	Drizzle ORM (drizzle-orm/node-postgres) sobre PostgreSQL — sem RLS nativo
Autenticação	better-auth (email+senha, Google OAuth, 2FA/TOTP, plugin admin)
Cache / Fila	Redis (ioredis)
Armazenamento	MinIO (S3-compatible) — buckets privados com URL assinada
Frontend	Next.js (App Router) + React, Tailwind, componentes @app/ui
E-mail	Resend + @react-email/components (templates JSX)
IA / Visão	Google Gemini (geração de questões) + OpenCV/pdfjs (leitura de gabarito escaneado)
Deploy local	docker-compose.dev.yml (Postgres, Redis, MinIO) — sem CI/CD (.github/workflows) no repo

Nota metodológica — mapeamento das categorias para esta stack

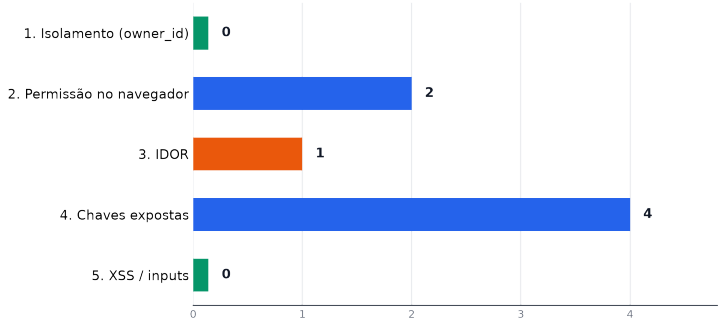
Categoria solicitada	Equivalente mapeado nesta stack
1. Banco sem tranca (isolamento)	Projeto não usa Supabase/RLS. O mecanismo de isolamento é manual: cada query em apps/api/src/modules deve filtrar por teacherId (turmas) ou creatorId (provas), derivados da sessão better-auth via getAuthenticatedUser(request) — nunca de parâmetro de URL/body.
2. Permissão definida no navegador	Único papel do sistema é o admin do plugin better-auth (session.user.role). Verificado se toda rota /admin/* e /api/auth/admin/* revalida role==='admin' (+2FA +sessão fresca) no servidor, e não apenas oculta UI no Next.js.
3. IDOR	Todo handler Fastify que recebe um :id (params) foi percorrido individualmente para confirmar que o recurso pai E os recursos aninhados (ex.: opção de questão dentro de uma questão) são validados contra o dono autenticado antes de leitura/escrita/exclusão.
4. Chaves expostas	.env / .env.example / docker-compose.dev.yml / packages/env (validação Zod de startup) / histórico Git (git log -p) / busca por padrões de chave (Google, Resend, Gemini) em todo o código rastreado.
5. Inputs sem tratamento (XSS)	React (JSX) escapa por padrão; procurou-se por dangerouslySetInnerHTML, innerHTML, eval, renderização de markdown sem sanitização e href/src com URL controlada pelo usuário, tanto no frontend quanto nos templates de e-mail do backend (@react-email/components).

1. Resumo Executivo

A auditoria percorreu, arquivo por arquivo, os 5 módulos de domínio do backend (**classrooms**, **exams**, **submissions**, **subjects**, **admin**, **auth**), toda a camada de autenticação/autorização (better-auth, guarda de admin, 2FA), a camada de armazenamento (MinIO), os templates de e-mail, a validação de ambiente e todo o frontend em busca de sinks de HTML não sanitizado. Foram identificados **7 achados** — nenhum crítico, 1 de severidade alta (falha de autorização de objeto aninhado) e 6 de severidade baixa/informativa — sobre uma base de código que aplica de forma consistente o padrão de verificação de posse (*ownership check*) em praticamente toda a superfície de API.



Achados por severidade



Achados por categoria (1–5, conforme escopo solicitado)

Severidade	Qtde.	Descrição
CRÍTICA	0	Nenhum achado crítico (sem RCE, sem bypass total de autenticação, sem exposição pública de dados de terceiros).
ALTA	1	Falha de autorização de objeto aninhado (IDOR) em edição de alternativa de questão.
MÉDIA	0	Nenhum achado de severidade média.
BAIXA	3	Guarda de admin por handler sem hook central; defaults inseguros no docker-compose de desenvolvimento; validação de startup não rejeita segredos padrão conhecidos em produção.
INFORMATIVA	3	Higiene de segredos (.env local e .env.example) e ausência de checagem de sessão no servidor para o layout privado do dashboard (a API já protege os dados de qualquer forma).

2. Pontos Fortes e Pontos Fracos

2.1 Pontos fortes (verificados, com evidência)

✓ Isolamento por dono aplicado em 100% das rotas de leitura/listagem

Todos os handlers de **classrooms**, **exams** e **submissions** (create, get, list, update, delete — 24 arquivos lidos integralmente) filtram explicitamente por teacherId/creatorId igual ao usuário autenticado, incluindo quando o dado vem de cache Redis (o campo de posse é revalidado após a leitura do cache). Ex.: apps/api/src/modules/classrooms/list-classrooms.ts:22-23, apps/api/src/modules/exams/get-exam.ts:22, apps/api/src/modules/submissions/fetch-owned-submission.ts:14.

✓ Guarda de admin consistente (8/8 rotas) com defesa em profundidade

Todo handler em apps/api/src/modules/admin/*.ts chama getAuthenticatedAdmin(request) como primeira linha, que verifica role==='admin', 2FA ativo e frescor de sessão (≤30 min) — apps/api/src/lib/auth/get-authenticated-admin.ts:18-38. As rotas nativas do plugin admin do better-auth (/api/auth/admin/*, que por padrão só teriam o check de role da própria lib) recebem uma guarda adicional explícita em apps/api/src/app.ts:79-102 + apps/api/src/lib/auth/guard-admin-auth-routes.ts, fechando um gap documentado no próprio comentário do código.

✓ Nenhum sink de HTML não sanitizado no frontend

Busca exaustiva por dangerouslySetInnerHTML, innerHTML, __html, v-html, eval(e document.write em apps/web/src, packages/ui e packages/shared não retornou nenhuma ocorrência. Não há biblioteca de markdown/HTML rendering no projeto. Toda a interpolação de dados do usuário passa por JSX (auto-escapado).

✓ Templates de e-mail seguros por construção

Os 5 templates em apps/api/src/lib/mail/templates/*.tsx usam @react-email/components — texto do usuário (nome, dispositivo, IP) é sempre interpolado como filho de <Text>, nunca concatenado em HTML bruto.

✓ Segredos carregados só de variável de ambiente, com validação de startup

packages/env/src/index.ts:29 exige BETTER_AUTH_SECRET (min 1 caractere, sem valor default) via schema Zod; o processo falha ao iniciar (dbEnvSchema.parse / apiEnvSchema.parse) se DATABASE_URL, REDIS_URL ou BETTER_AUTH_SECRET estiverem ausentes. Nenhum fallback tipo process.env.X || 'valor-fixo' foi encontrado para segredos de produção.

✓ .env nunca foi versionado

.env está listado em .gitignore:69 e `git ls-files` confirma que só .env.example está rastreado; `git log --all --diff-filter=A -- .env` não retornou nenhum commit em todo o histórico.

✓ Revogação de sessão validada no lado do servidor pela própria lib de auth

revoke-session.ts encaminha o token para better-auth; o handler da lib (node_modules/better-auth/dist/api/routes/session.mjs:398-400) só apaga a sessão se session.userId === ctx.context.session.user.id — um usuário não consegue revogar sessão de outro.

✓ Object storage de dados sensíveis é privado, com URL assinada de curta duração

A policy pública do bucket MinIO (apps/api/src/lib/storage/storage.ts:24-37) libera GetObject apenas para o prefixo avatars/*. Fotos de folhas de resposta (submission-pages/*) e PDFs de gabarito travado (exam-templates/*) não têm policy pública — o acesso é sempre via presignedGetObject de 1 hora, gerado só depois do ownership check em fetchOwnedSubmission.

✓ Proteção contra open redirect no login social e nenhuma injeção de SQL

signInWithSocialSchema (apps/api/src/modules/auth/sign-in-with-social-schema.ts:6-18) restringe callbackURL à origem de BETTER_AUTH_URL. Nenhuma query com sql.raw ou interpolação de string foi encontrada em todo apps/api/src — 100% das queries usam o construtor Drizzle parametrizado (eq, inArray etc.).

✓ Script de seed recusa rodar em produção

apps/api/src/scripts/seed.ts:135-140 verifica env.NODE_ENV === 'prod' e aborta o processo (process.exit(1)) antes de criar contas de teste com senha fixa conhecida — o único lugar do código que já implementa exatamente o tipo de guarda de startup recomendado na seção 4 para os demais segredos padrão do projeto.

2.2 Pontos fracos (risco central)

✗ Autorização de objeto aninhado quebrada em 1 endpoint de escrita

apps/api/src/modules/exams/update-question.ts valida a posse da *prova* e da *questão*, mas atualiza cada question_options usando o id enviado no corpo da requisição sem confirmar que essa opção pertence à questão validada — permitindo, em tese, sobrescrever a alternativa de outro professor. Ver achado detalhado na seção 4.

✗ Guarda de admin depende de disciplina por arquivo, não de um hook central

Hoje a cobertura é 8/8, mas nada no roteador (apps/api/src/modules/admin/admin-routes.ts) obriga uma nova rota futura a chamar getAuthenticatedAdmin — o padrão já usado no catch-all /api/auth/admin/* (preHandler central) não foi replicado para /admin/*.

✗ Segredos de terceiros vivos em texto plano em .env local + defaults fracos herdáveis

O .env local contém chaves de API que parecem válidas (Google OAuth, Resend, Gemini, BETTER_AUTH_SECRET); embora nunca commitadas, isso é um ponto de exposição caso o arquivo seja copiado manualmente. docker-compose.dev.yml e .env.example também usam credenciais padrão triviais (postgres/postgres, minioadmin/minioadmin) sem aviso explícito de "nunca usar em produção".

✗ Validação de ambiente não rejeita segredos padrão conhecidos, mesmo em produção

packages/env/src/index.ts:15-52 garante presença/formato das variáveis via Zod, mas não recusa valores iguais aos defaults de desenvolvimento (ex.: MINIO_SECRET_KEY='minioadmin', POSTGRES_PASSWORD='postgres') quando NODE_ENV='prod' — diferente do próprio apps/api/src/scripts/seed.ts, que já implementa esse tipo de guarda para a senha de seed.

3. Achados Detalhados por Categoria

3.1 Categoria 1 — Banco sem tranca (isolamento de dono)

Nenhum achado. Não aplicável no formato Supabase/RLS: o projeto usa PostgreSQL puro via Drizzle ORM, sem políticas RLS. O mecanismo de isolamento equivalente é o filtro manual por teacherId/creatorId em cada query, obtido de getAuthenticatedUser(request) — nunca de um parâmetro de URL. Foram lidos integralmente os 24 arquivos de caso de uso de classrooms/exams/submissions e todos aplicam o filtro corretamente em toda operação de listagem, agregação e exportação (ver evidências na seção 2.1). O único gap de autorização encontrado no projeto é de granularidade mais fina (objeto aninhado dentro de um recurso já validado) e está classificado na categoria 3 — IDOR, abaixo.

3.2 Categoria 2 — Permissão definida no navegador

BAIXA **Checagem de privilégio de admin é replicada por handler, sem hook central no roteador**

apps/api/src/modules/admin/admin-routes.ts (todas as rotas) · apps/api/src/lib/auth/get-authenticated-admin.ts:18 · categoria: 2. Permissão no navegador / arquitetura de autorização

O frontend oculta a navegação e as páginas de admin (apps/web/src/app/(admin)/admin/layout.tsx:20 e 24) e o backend **revalida corretamente** em todos os 8 handlers atuais de apps/api/src/modules/admin/*.ts, cada um chamando await getAuthenticatedAdmin(request) como primeira instrução. Isso não é, hoje, um bypass — é uma fragilidade estrutural: admin-routes.ts registra as rotas sem nenhum preHandler que force essa checagem, então a proteção depende inteiramente de cada novo arquivo lembrar de chamar a função manualmente. O próprio projeto já reconhece esse risco: o catch-all /api/auth/admin/* (rotas nativas do plugin admin do better-auth) recebe uma guarda centralizada em apps/api/src/app.ts:79-102 exatamente para não depender de disciplina por handler — o mesmo padrão não foi replicado para as rotas de negócio em /admin/*.

```
// admin-routes.ts registra as rotas sem preHandler central:
export function adminRoutes(app: FastifyInstance) {
app.get('/overview', ADMIN_ROUTE_OPTIONS, getAdminOverviewModule)
// ... nenhuma verificação aqui; getAuthenticatedAdmin() só roda
// porque CADA módulo chama a função manualmente na 1a linha
```

Por que é explorável: Não há exploit hoje (cobertura 8/8 confirmada por leitura integral de todos os arquivos do módulo). O risco é de regressão: uma rota futura adicionada a admin-routes.ts sem a chamada manual ficaria acessível a qualquer usuário autenticado (ou até anônimo) sem que nenhum teste de integração aponte isso automaticamente, pois nada no roteador Fastify reforça o requisito.

Condição de explorabilidade: Só vira exploit se uma rota nova for adicionada sem chamar getAuthenticatedAdmin — não há evidência de que isso já ocorreu.

INFORMATIVA **Layout do dashboard privado não repete a checagem de sessão no servidor feita pelo layout de admin**

apps/web/src/app/(private)/layout.tsx:1-23 · categoria: 2. Permissão no navegador

Diferente de apps/web/src/app/(admin)/admin/layout.tsx (que chama getAuthSession() e redireciona no servidor antes de renderizar), o layout de (private) — usado pelas páginas de turmas, provas e submissões do professor comum — não faz nenhuma verificação de sessão; ele apenas monta o shell (sidebar) e renderiza os filhos diretamente.

```
export default function PrivateLayout({ children }: { children: React.ReactNode }) {
return (


{children}

)
}
```

Por que é explorável: Não expõe dado nenhum: toda chamada de dados do dashboard passa por apps/web/src/lib/api-client.ts com credentials: 'include', e o backend autentica cada requisição de forma independente via getAuthenticatedUser(request) (confirmado em todos os handlers de classrooms/exams/submissions na seção 2.1). Um visitante não autenticado só veria o esqueleto vazio da página (sidebar sem dados), nunca informação de outro usuário.

Condição de explorabilidade: Puramente uma inconsistência de defesa em profundidade / UX (flash do shell antes do redirecionamento client-side em caso de 401) — não é, por si só, explorável.

3.3 Categoria 3 — IDOR (Insecure Direct Object Reference)

ALTA **Atualização de alternativa (question_options) por ID do corpo sem validar posse em relação à questão**

apps/api/src/modules/exams/update-question.ts:51-59 · categoria: 3. IDOR

O handler valida corretamente que examId pertence ao professor autenticado (linha 32) e que questionId pertence a esse examId (linha 41). Porém, ao aplicar as alterações de cada alternativa, o loop usa o option.id vindo do corpo da requisição diretamente na cláusula WHERE, sem checar se esse option.id realmente pertence à questionId já validada — diferente do padrão correto aplicado em update-correct-option.ts:47-53, que explicitamente rejeita a requisição se option.questionId !== questionId.

```
// update-question.ts:51-59
for (const [index, option] of options.entries()) {
await tx
.update(questionOptionsTable)
.set({
text: option.text,
letter: letterForOptionIndex(question.type, index),
})
.where(eq(questionOptionsTable.id, option.id)) // <- sem checar dono
}

// Contraste — o padrão CORRETO, já usado em update-correct-option.ts:47-53:
const [option] = await db.select().from(questionOptionsTable)
.where(eq(questionOptionsTable.id, optionId))
if (!option || option.questionId !== questionId) {
throw new NotFoundError('Alternativa não encontrada.')
}
```

Por que é explorável: Um professor autenticado (qualquer conta, a barreira de entrada é só se cadastrar) pode enviar PATCH /exams/<prova-própria>/questions/<questão-própria> com corpo { statement: '...', options: [{ id: '<uuid-de-outra-conta>', text: 'texto malicioso' }] }. A query final executa UPDATE question_options SET text=..., letter=... WHERE id = <uuid alheio>, sobrescrevendo o texto/letra de uma alternativa de **outro professor**, mesmo sem qualquer relação entre as duas contas. O schema de validação (packages/shared/.../update-question-schema.ts:17) só exige que id seja um UUID válido — não valida propriedade.

Condição de explorabilidade: Exige conhecer (ou obter em algum outro fluxo) o UUID de uma question_option de outra conta — não há, hoje, nenhum endpoint do sistema que vaze esse UUID para um usuário não-dono. Ou seja: é uma falha real de autorização, mas a exploração prática atual depende de um vetor secundário de vazamento de ID (logs, integração futura, força bruta de UUID — inviável em escala). Deve ser corrigida independente da dificuldade atual de exploração, pois qualquer novo endpoint que devolva IDs de outras contas (ex.: exportação, integração, webhook) tornaria o ataque diretamente utilizável.

3.4 Categoria 4 — Chaves expostas (hardcode)

BAIXA **Defaults inseguros herdáveis (${VAR:-default}) sem aviso de "não usar em produção"**

docker-compose.dev.yml:7-8, 45-46 · categoria: 4. Chaves expostas

docker-compose.dev.yml usa fallback do tipo ${POSTGRES_PASSWORD:-postgres} e ${MINIO_ROOT_USER:-minioadmin}/${MINIO_ROOT_PASSWORD:-minioadmin}. O arquivo é nomeado explicitamente "dev" e não há indício de que seja usado como artefato de deploy em produção (não há docker-compose.prod.yml nem pipeline de CI/CD no repositório). O risco é organizacional: nada impede alguém de reaproveitar este arquivo como base para um ambiente publicamente acessível sem trocar os defaults.

```
POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-postgres}
MINIO_ROOT_USER: ${MINIO_ROOT_USER:-minioadmin}
MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD:-minioadmin}
```

Por que é explorável: Só é explorável se este compose (ou seus defaults) for promovido a um ambiente acessível pela rede sem sobrescrever as variáveis.

Condição de explorabilidade: Requer decisão operacional futura de reusar o compose de dev fora de localhost.

INFORMATIVA **Credenciais padrão do MinIO como valor literal, em vez de placeholder vazio**

.env.example:35-36, 41-42 · categoria: 4. Chaves expostas

Diferente das demais chaves sensíveis do arquivo (RESEND_API_KEY, GEMINI_API_KEY, GOOGLE_CLIENT_SECRET, que corretamente ficam em branco em .env.example), MINIO_ROOT_USER/MINIO_ACCESS_KEY/MINIO_SECRET_KEY já vêm preenchidos com minioadmin. Isso reduz o atrito para alguém copiar .env.example → .env e subir um ambiente com credencial pública e amplamente conhecida sem perceber.

Condição de explorabilidade: Baixo risco isolado; some ao achado anterior (docker-compose) se o mesmo ambiente reutilizar ambos.

INFORMATIVA **Higiene de segredos: .env local contém chaves aparentemente válidas de serviços de terceiros**

.env (arquivo local, não versionado) · categoria: 4. Chaves expostas

O arquivo .env local (confirmado fora do controle de versão — .gitignore:69-71 e git log --all --diff-filter=A -- .env sem nenhum resultado em todo o histórico) contém valores com formato de chave real para BETTER_AUTH_SECRET, GOOGLE_CLIENT_SECRET, RESEND_API_KEY e GEMINI_API_KEY. Os valores não são reproduzidos neste relatório. Não é uma vulnerabilidade de código, mas uma recomendação de higiene: se este arquivo já foi colado em algum chat, ticket ou repositório secundário, as chaves devem ser rotacionadas; para times, prefira um cofre de segredos (1Password, Doppler, Vault) em vez de compartilhar .env por fora do Git.

BAIXA **Validação de startup não rejeita valores de segredo iguais aos defaults conhecidos de desenvolvimento**

packages/env/src/index.ts:15-52 · categoria: 4. Chaves expostas

O schema Zod de ambiente (apiEnvSchema) garante que as variáveis existam e tenham o formato certo, mas não recusa iniciar quando um valor bate com um default público conhecido (MINIO_SECRET_KEY='minioadmin', POSTGRES_PASSWORD='postgres', o BETTER_AUTH_SECRET='minha-chave-secreta-dev' de .env.example) mesmo com NODE_ENV='prod'. Contraste direto com apps/api/src/scripts/seed.ts:135-140, que já implementa exatamente esse tipo de guarda (recusa rodar com senha de teste fixa quando NODE_ENV='prod').

```
export const apiEnv = apiEnvSchema.parse(process.env)
// nenhuma checagem de "valor conhecido como inseguro" é feita aqui,
// diferente do guard explícito já usado em scripts/seed.ts:135-140
```

Por que é explorável: Amplia a janela do achado anterior (docker-compose): se um deploy real herdar esses defaults, nada no boot da aplicação recusa subir.

Condição de explorabilidade: Só se materializa combinado a um erro operacional de deploy com as variáveis de dev.

3.5 Categoria 5 — Inputs sem tratamento (XSS)

Nenhum achado. Busca exaustiva (grep recursivo) por dangerouslySetInnerHTML, innerHTML, __html, eval(, new Function(e document.write em apps/web/src, packages/ui/src e packages/shared/src não encontrou nenhuma ocorrência. Não há biblioteca de renderização de markdown/HTML no projeto (react-markdown, rehype, remark) nem DOMPurify/sanitize-html — porque não há, hoje, nenhum ponto que precise deles: toda saída de dados do usuário passa por nós de texto JSX (auto-escapados pelo React) ou por componentes @react-email/components no backend (também JSX, também escapados). Todos os usos de atributo href dinâmico encontrados (apps/web/src/components/dashboard/*.tsx, admin-sidebar.tsx) constroem rotas internas do Next.js a partir de UUIDs da própria API — não há campo de texto livre do usuário alimentando um atributo href/src. Não aplicável / sem vetor de exploração identificado.

4. Recomendações Priorizadas

P1 **Corrigir o IDOR em update-question.ts**

Antes do UPDATE de cada alternativa, buscar as question_options existentes da questão (uma única query com WHERE questionId = :questionId) e validar que todo option.id do corpo pertence a esse conjunto — rejeitando a requisição inteira (400/404) caso qualquer ID não pertença. Replicar o padrão já usado em update-correct-option.ts:47-53.

P2 **Centralizar a guarda de admin com um preHandler no registro das rotas**

Registrar getAuthenticatedAdmin como preHandler na chamada app.register(adminRoutes, { prefix: '/admin' }) (ou via onRequest hook dentro de adminRoutes), como já é feito para /api/auth/admin/* em app.ts. Isso torna estruturalmente impossível esquecer a checagem numa rota futura, mesmo que os handlers atuais já a chamem corretamente.

P3 **Adicionar avisos explícitos de "não usar em produção" aos defaults de desenvolvimento**

Adicionar comentário no topo de docker-compose.dev.yml e trocar os valores literais de MINIO_ROOT_USER/MINIO_ACCESS_KEY/MINIO_SECRET_KEY em .env.example por placeholders vazios ou claramente marcados como "trocar antes de expor a rede", alinhando com o padrão já usado para RESEND_API_KEY/GEMINI_API_KEY.

P4 **Replicar em packages/env o guard de startup que já existe em scripts/seed.ts**

Adicionar um refine/checagem pós-parse em apiEnvSchema que falhe o boot quando NODE_ENV='prod' e algum segredo bater com um valor padrão conhecido (postgres, minioadmin, minha-chave-secreta-dev) — mesmo espírito da guarda que já existe em apps/api/src/scripts/seed.ts:135-140 para a senha de seed.

P5 **Adotar um cofre de segredos para o time e rotacionar chaves que já circularam fora do Git**

Migrar .env de desenvolvimento compartilhado para um gerenciador de segredos (1Password, Doppler, Vault) e rotacionar BETTER_AUTH_SECRET/GOOGLE_CLIENT_SECRET/RESEND_API_KEY/GEMINI_API_KEY caso o arquivo .env local já tenha sido compartilhado por qualquer canal fora do controle de versão. Opcionalmente, replicar em apps/web/src/app/(private)/layout.tsx a mesma checagem de sessão server-side já usada no layout de admin, por consistência de defesa em profundidade.

5. Issues para o GitHub

Texto completo, em Markdown, pronto para copiar e colar em cada issue. Achados triviais e relacionados do tema "defaults inseguros de desenvolvimento" foram agrupados em uma única issue para evitar spam.

--- ISSUE 1 ---

````
# [Segurança] IDOR: update-question permite sobrescrever alternativa de outro professor

**Labels sugeridas:** security, severity:high

## Descrição do problema
`updateQuestionModule` (apps/api/src/modules/exams/update-question.ts) valida que `examId` e
`questionId`
pertencem ao professor autenticado, mas ao atualizar cada `question_options` usa o `option.id`
recebido no
corpo da requisição diretamente na cláusula WHERE, sem confirmar que essa opção pertence a
`questionId`.
Isso quebra o modelo de autorização por posse (ownership) usado em todo o resto do backend.

## Evidência
apps/api/src/modules/exams/update-question.ts:51-59
```ts
for (const [index, option] of options.entries()) {
await tx
.update(questionOptionsTable)
.set({
text: option.text,
letter: letterForOptionIndex(question.type, index),
})
.where(eq(questionOptionsTable.id, option.id))
}
```
Compare com o padrão correto em apps/api/src/modules/exams/update-correct-option.ts:47-53, que
explicitamente
rejeita `option.questionId !== questionId`.

## Impacto
Um professor autenticado que conheça (ou consiga, por qualquer vetor futuro) o UUID de uma
`question_option`
de outra conta pode sobrescrever seu texto/letra via PATCH em uma prova/questão que ele próprio
possui.
Corrupção de dados de terceiros sem relação com a conta atacante.

## Sugestão de correção
Antes do loop de update, buscar todas as `question_options` da `questionId` já validada e checar
que
`options.every(o => existingIds.has(o.id))`; rejeitar com 404/400 caso contrário. Reaproveitar
exatamente o
padrão de `update-correct-option.ts`.

## Critérios de aceite
- [ ] PATCH /exams/:examId/questions/:questionId com um `options[].id` que não pertence a
`questionId` retorna 404 (ou 400) e não altera nenhuma linha no banco
- [ ] Teste de integração cobrindo o caso acima adicionado em update-question.spec.ts
- [ ] Fluxo legítimo (todas as opções pertencentes à questão) continua funcionando sem regressão
````

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

```
# [Segurança] Guarda de admin sem preHandler central no roteador /admin

**Labels sugeridas:** security, severity:low, tech-debt

## Descrição do problema
Cada handler em apps/api/src/modules/admin/*.ts chama `getAuthenticatedAdmin(request)` manualmente
como
primeira linha. Hoje a cobertura é 8/8, mas não há nenhum preHandler/onRequest no registro das
rotas
(admin-routes.ts) que torne essa checagem obrigatória — uma rota nova poderia ser adicionada sem a
chamada e
ficaria acessível sem verificação de privilégio. O catch-all /api/auth/admin/* já usa o padrão
correto
(guarda centralizada em app.ts:79-102) e deveria ser replicado aqui.

## Evidência
apps/api/src/modules/admin/admin-routes.ts (registro de rotas sem preHandler)
apps/api/src/lib/auth/get-authenticated-admin.ts:18-38 (função chamada manualmente em cada
handler)

## Impacto
Nenhum hoje (cobertura confirmada). Risco de regressão silenciosa em features futuras.

## Sugestão de correção
Adicionar `getAuthenticatedAdmin` como preHandler no `app.register(adminRoutes, { prefix: '/admin'
})` em
app.ts, ou como `onRequest` hook dentro de `adminRoutes()`.

## Critérios de aceite
- [ ] Uma rota admin hipotética sem chamada manual a getAuthenticatedAdmin ainda assim exige
role==='admin' + 2FA + sessão fresca
- [ ] Handlers existentes continuam funcionando (chamada manual duplicada não quebra nada, ou é
removida)
```

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

```
# [Segurança] Defaults inseguros de desenvolvimento sem aviso (docker-compose + .env.example)

**Labels sugeridas:** security, severity:low, documentation

## Descrição do problema
Agrupando dois achados triviais do mesmo tema — credenciais padrão previsíveis usadas como
fallback/exemplo:

1. docker-compose.dev.yml usa `${POSTGRES_PASSWORD:-postgres}`, `${MINIO_ROOT_USER:-minioadmin}` e
`${MINIO_ROOT_PASSWORD:-minioadmin}` sem nenhum aviso de que esses defaults nunca devem chegar a um
ambiente acessível pela rede.
2. .env.example preenche MINIO_ROOT_USER/MINIO_ACCESS_KEY/MINIO_SECRET_KEY com o valor literal
`minioadmin`,
diferente das demais chaves sensíveis do arquivo (RESEND_API_KEY, GEMINI_API_KEY,
GOOGLE_CLIENT_SECRET),
que corretamente ficam em branco.

## Evidência
docker-compose.dev.yml:7-8, 45-46
.env.example:35-36, 41-42

## Impacto
Baixo isoladamente (arquivo é nomeado "dev", sem CI/CD ou compose de produção no repo). Torna-se
relevante
apenas se alguém reusar este arquivo/valores como base para um ambiente exposto publicamente.

## Sugestão de correção
- Adicionar comentário no topo de docker-compose.dev.yml: "Apenas para desenvolvimento local —
nunca expor estas credenciais em rede pública."
- Trocar os valores literais de MINIO_* em .env.example por placeholder vazio ou por um valor
claramente marcado como "trocar antes de subir".

## Critérios de aceite
- [ ] docker-compose.dev.yml tem aviso explícito no cabeçalho
- [ ] .env.example não contém nenhuma credencial MinIO literal reaproveitável
```

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

```
# [Segurança] Validação de ambiente não rejeita segredos padrão conhecidos em produção

**Labels sugeridas:** security, severity:low

## Descrição do problema
`apiEnvSchema` (packages/env/src/index.ts:15-52) garante presença e formato das variáveis via Zod,
mas não
recusa iniciar quando um valor bate com um default público conhecido
(`MINIO_SECRET_KEY='minioadmin'`,
`POSTGRES_PASSWORD='postgres'`, `BETTER_AUTH_SECRET='minha-chave-secreta-dev'` de .env.example)
mesmo com
`NODE_ENV='prod'`. O próprio projeto já implementa esse tipo de guarda em outro lugar
(apps/api/src/scripts/seed.ts:135-140, que recusa rodar com senha de seed fixa quando
NODE_ENV='prod') — só
não foi replicado para o carregamento de ambiente da API.

## Evidência
packages/env/src/index.ts:15-52
```ts
export const apiEnv = apiEnvSchema.parse(process.env)
// nenhuma checagem de "valor conhecido como inseguro" é feita aqui,
// diferente do guard explícito já usado em scripts/seed.ts:135-140
```

Impacto
Amplia a janela de exploração de qualquer segredo/credencial default (ex.: docker-compose.dev.yml,
ver issue
de defaults inseguros): se um deploy real herdar essas variáveis por engano, nada no boot da
aplicação recusa
subir com elas.

Sugestão de correção
Adicionar um `refine`/checagem pós-parse em `apiEnvSchema` que falhe o boot (`process.exit(1)` ou
lançar) quando
`NODE_ENV==='prod'` e algum segredo bater com uma lista de valores padrão conhecidos (`postgres`,
`minioadmin`,
`minha-chave-secreta-dev`), no mesmo espírito da guarda já existente em
`apps/api/src/scripts/seed.ts:135-140`.

Critérios de aceite
- [ ] Com NODE_ENV='prod' e qualquer segredo monitorado igual a um valor padrão conhecido, o
processo falha ao iniciar com uma mensagem clara
- [ ] Com NODE_ENV!=='prod', o comportamento atual (sem restrição) é preservado
- [ ] Teste unitário cobrindo ao menos um caso de rejeição e um caso de boot normal
```

--- FIM ISSUE 4 ---